

STAF Framework — Architecture, Features & User Guide

Document version 4.4.0 · Last updated March 2026

1. What Is STAF and How It Helps You

STAF (Simple Test Automation Framework) is a .NET test automation framework that lets you run UI tests (Selenium), API tests, Excel checks, and accessibility scans in one place, with one reporting model.

- **One stack:** UI, API, Excel, database, and accessibility in a single framework — no need to glue different tools together.
- **Less boilerplate:** Base classes (TestBaseClass, PageBaseClass) handle driver setup, waits, and cleanup so you focus on test logic.
- **Built-in reporting:** Every test step can be recorded as Pass/Fail/Warn/Info, with per-test HTML reports and an assembly summary so you see results at a glance.
- **Easier adoption:** The sample project includes an optional MCP Agent so you can use AI (e.g., in Cursor or VS Code) to drive the browser and generate STAF-ready code without extra setup.

2. Features at a Glance

Feature	What you get
Base classes	TestBaseClass (UI), TestBaseAPI (API/Excel/DB), PageBaseClass with FindAppElement and built-in waits.
Browsers	Chrome, Edge; local or remote WebDriver; override options if needed.
HTML reporting	Step-by-step Pass/Fail/Warn/Info and a final assembly summary (e.g. ResultTemplateFinal.html).
Parallel execution	Run many tests in parallel with thread-safe reporting.
Excel	Compare workbooks/sheets and read/write cells (ClosedXML).
Database	DbHelper for SQL Server (connection from config, query/scalar/non-query).
Accessibility	Axe-core integration: full-page or scoped scans and HTML reports.
Configuration	appsettings.json and run settings for browser, URL, and test parameters.

3. Reporting — How Results Are Presented

Reporting is central to STAF so that end users and teams can quickly understand what passed or failed.

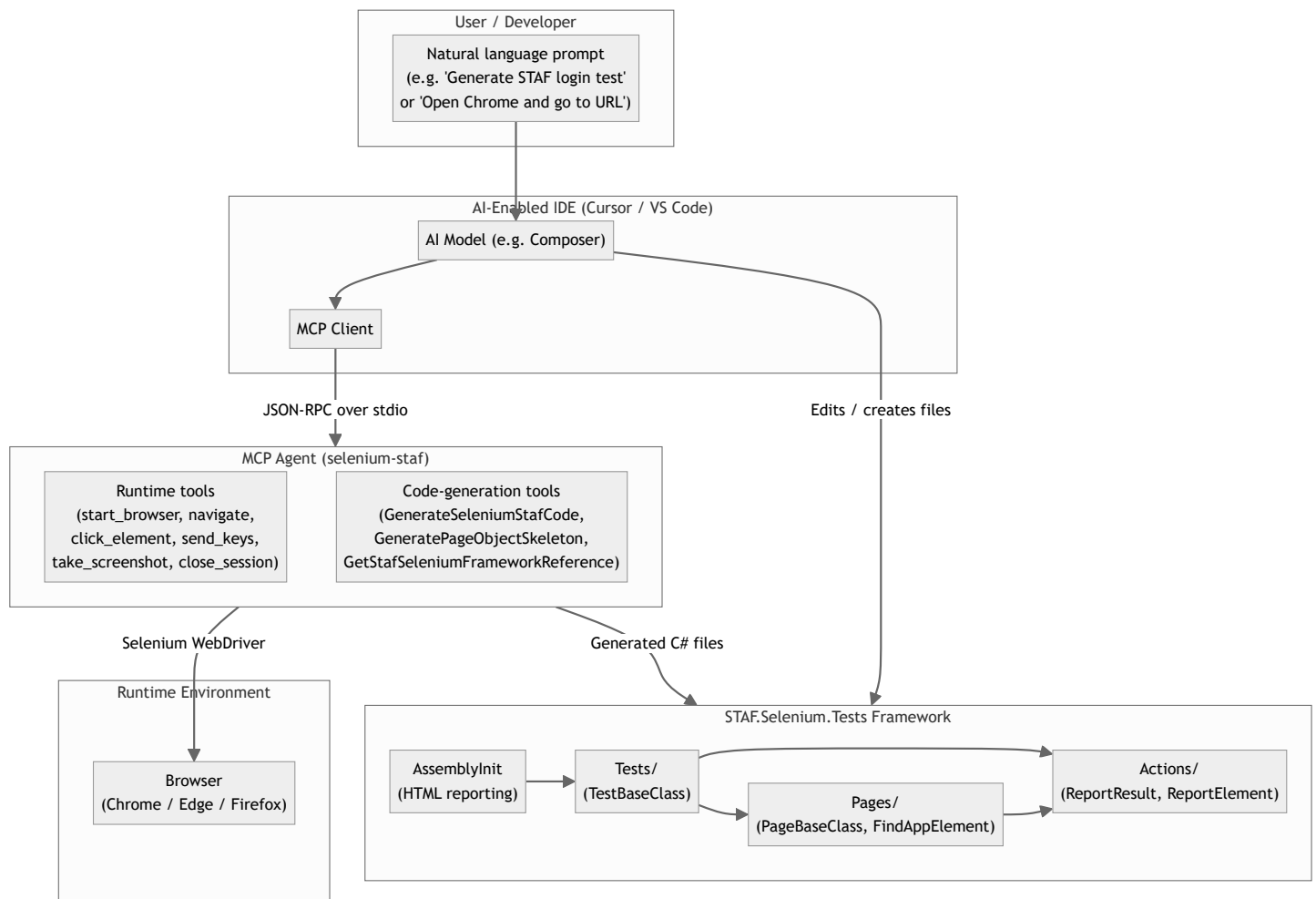
- **Per-test steps:** Each logical step (e.g. "Login button clicked", "API status check passed") is recorded with `ReportResult` or `ReportResultAPI`. You see a clear trail of actions and outcomes.
- **Per-test HTML file:** Every test gets its own HTML report. On failure, a screenshot can be attached automatically.
- **Assembly summary:** After all tests run, a single summary file (e.g. `ResultTemplateFinal.html`) aggregates results so you can review the full run at a glance.
- **Assertions and reporting together:** `ReportElement` extensions (e.g. `ReportElementIsDisplayed`) both assert and log in one call, so the report stays in sync with your checks without extra code.

4. Technical Architecture

The system is built in clear layers: you and the IDE talk to an optional MCP Agent, which can control a browser or generate STAF code. The test project runs on its own via MSTest and does not depend on the Agent at build or run time.

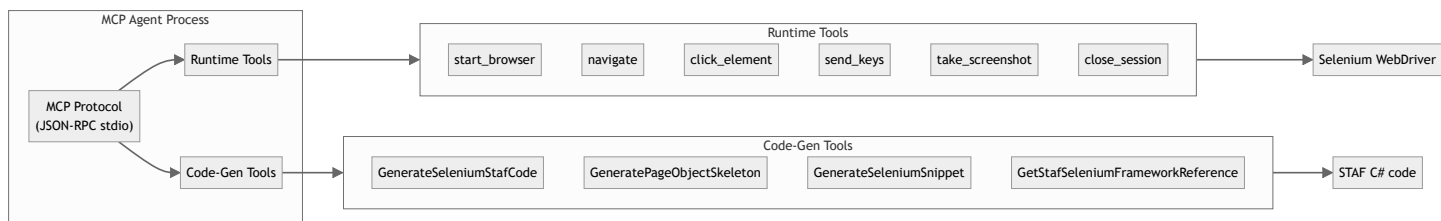
4.1 System Context: User → IDE → MCP Agent → Browser / Framework

Figure 1 — High-level flow



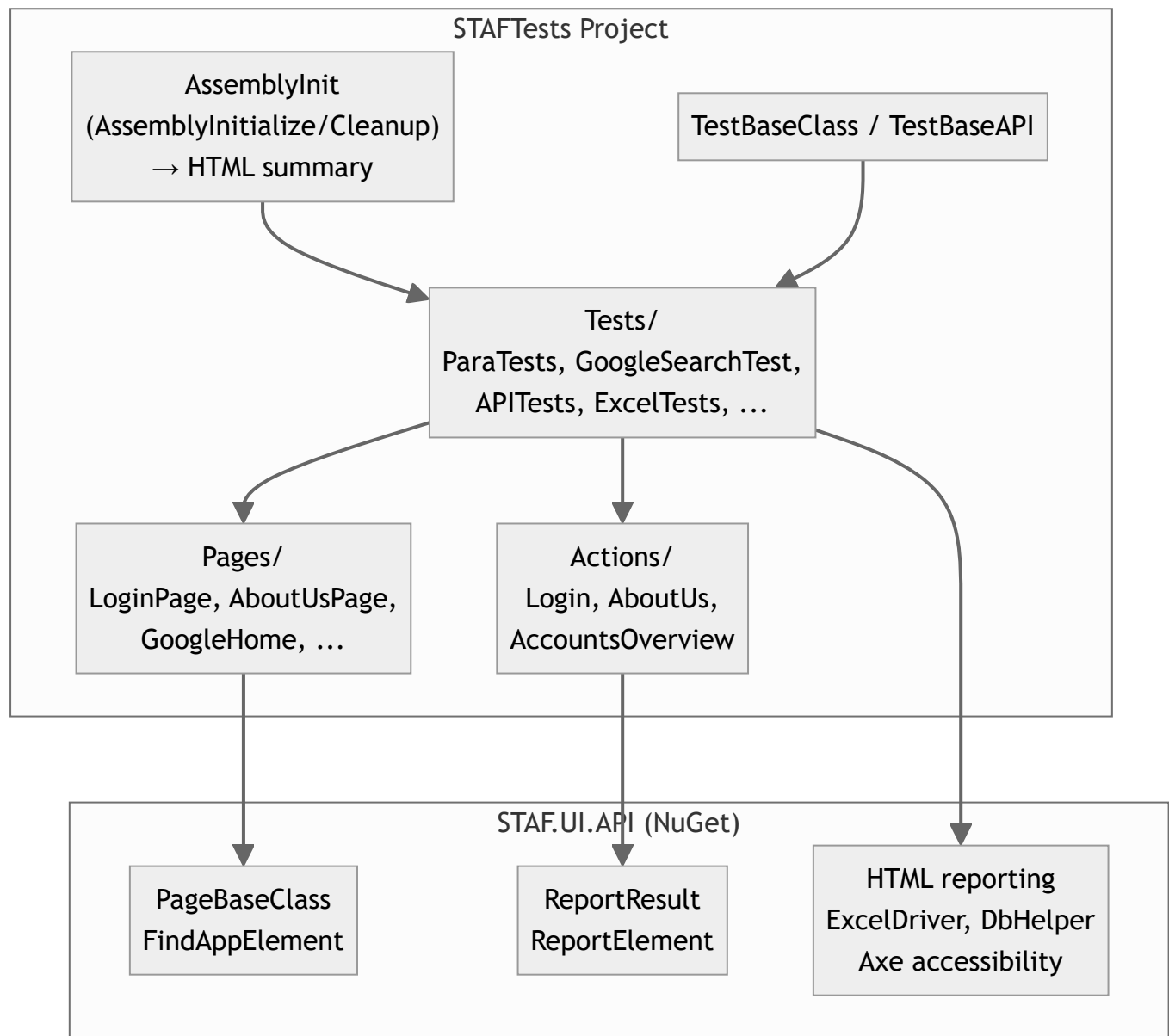
4.2 MCP Agent Internal View (Tools)

Figure 2 — MCP Agent tools



4.3 STAF Framework Structure (Test Project)

Figure 3 — Test project and STAF.UI.API



5. Why STAF Is Different (Novelty)

- **One MCP server, two roles:** The same MCP server can control the browser (navigate, click, type, screenshot) and generate STAF code (Pages, Actions, Tests). You can explore in the browser and then get framework-ready code without switching tools.

- **Framework-aware code generation:** Generated code uses PageBaseClass, FindAppElement, ReportResult, TestBaseClass, and the project folder layout — so it fits your reporting and patterns from the start.
- **Out-of-box AI integration:** The sample repo includes MCP config and a self-contained Agent executable. Clone, open in Cursor or VS Code, and use AI-driven browser control and code generation without extra installs.
- **Clear separation:** The MCP Agent is optional and separate from the test project. Tests run with MSTest (e.g. dotnet test or IDE) with or without the Agent.
- **Unified stack:** UI, API, reporting, and accessibility live in one framework and one package (STAF.UI.API), so one set of patterns and one place to look for results.

6. Key Components (Quick Reference)

Component	Purpose
TestBaseClass	Base for UI tests: driver setup, per-test HTML report, cleanup, assembly summary.
TestBaseAPI	Base for API/Excel/DB tests: same reporting and lifecycle, no browser.
PageBaseClass	FindAppElement with waits; no direct driver use in pages.
ReportResult / ReportResultAPI	Log steps (Pass/Fail/Warn/Info) for UI and API tests.
ReportElement	Assert and report in one call (e.g. ReportElementIsDisplayed).
BrowserDriver	Creates Chrome/Edge driver; override options if needed.

7. Getting Started and Links

Use the sample project to run UI, API, Excel, and accessibility examples. Configure run settings (e.g. testrunsetting.runsettings) for browser and URL, then run tests via Test Explorer or dotnet test.

STAF framework (GitHub): <https://github.com/sooraj171/STAF>

Sample project and MCP Agent (GitHub): <https://github.com/sooraj171/STAF.Selenium.Tests>